Project Overview (DoDay)

[Goal: Labor Costs reduced by 67% in DoDay]

Labor Intensive

Labor Innovation

High Error Rate

A typical store usually requires **3-4 people**. Labor costs can be up to 40% of the total cost

67% Less Labor

Low Error Rate

First automated cafe using intelligent systems and robotics. Only need **1 person** to manage.


Auto Dessert Maker


DoDay
Ordering

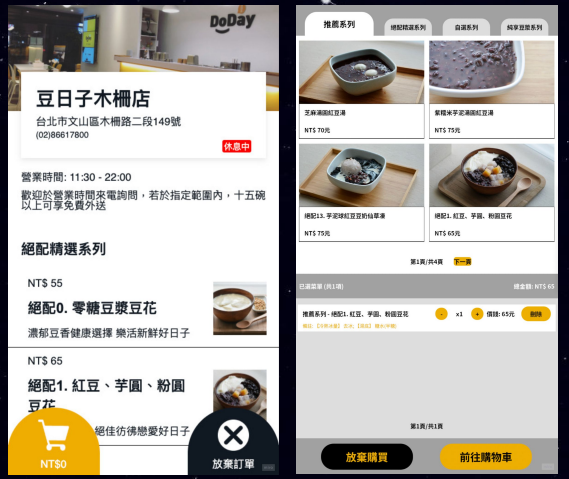

DoDay Deliver


portal.doday.com.tw/proj-002-doday-management-console/ManageTool/checkin
豆日子後台
首頁
總覽
庫存
管理
Home / 豆日子後台 / 簽到打卡
簽到打卡
1) 打卡頁面
請點擊出貨單取得你個人打卡專用二維碼。
2) 簽到紀錄
打卡紀錄
日期 工作時段 上班時間 下班時間 工作時數(h) 日累計時數(h)
2026-03-01 - - - 0 0
2026-03-02 - - - 0 0
2026-03-03 - - - 0 0
2026-03-04 - - - 0 0
2026-03-05 - - - 0 0
2026-03-06 - - - 0 0
2026-03-10 - - - 0 0
2026-03-11 - - - 0 0
DoDay Management


DoDay


DoDay
Design & Built DoDay

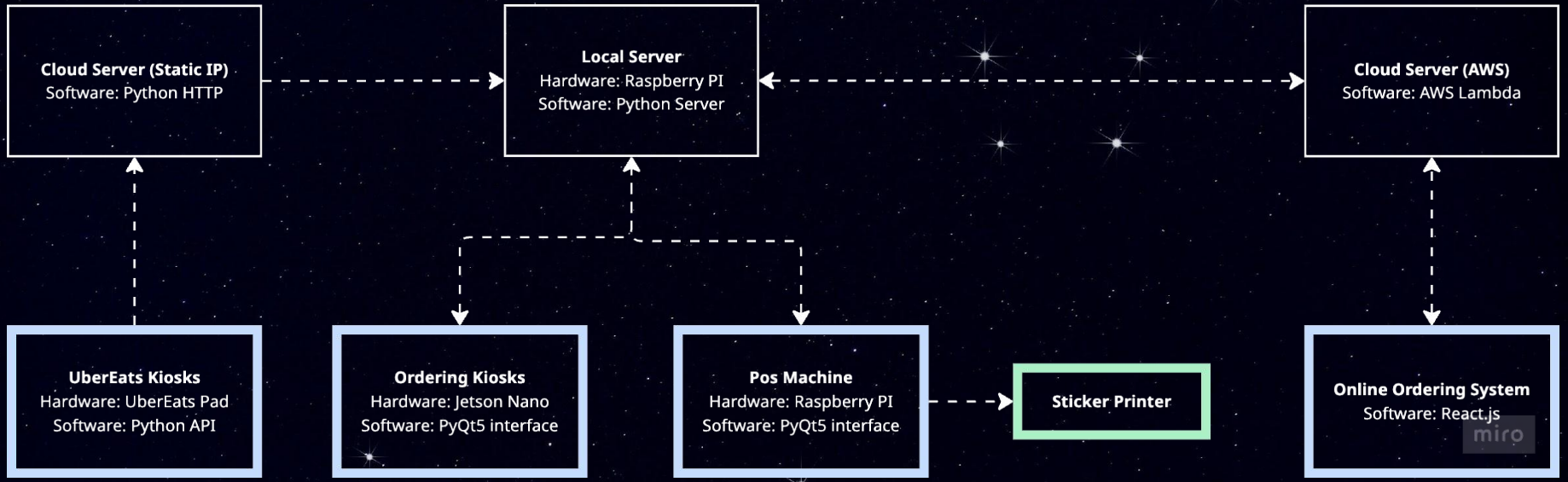
Project Overview (DoDay Ordering)



Problem: Staff Shortage + Complex Orders = Data Errors. To fix this, we must eliminate manual data entry entirely.

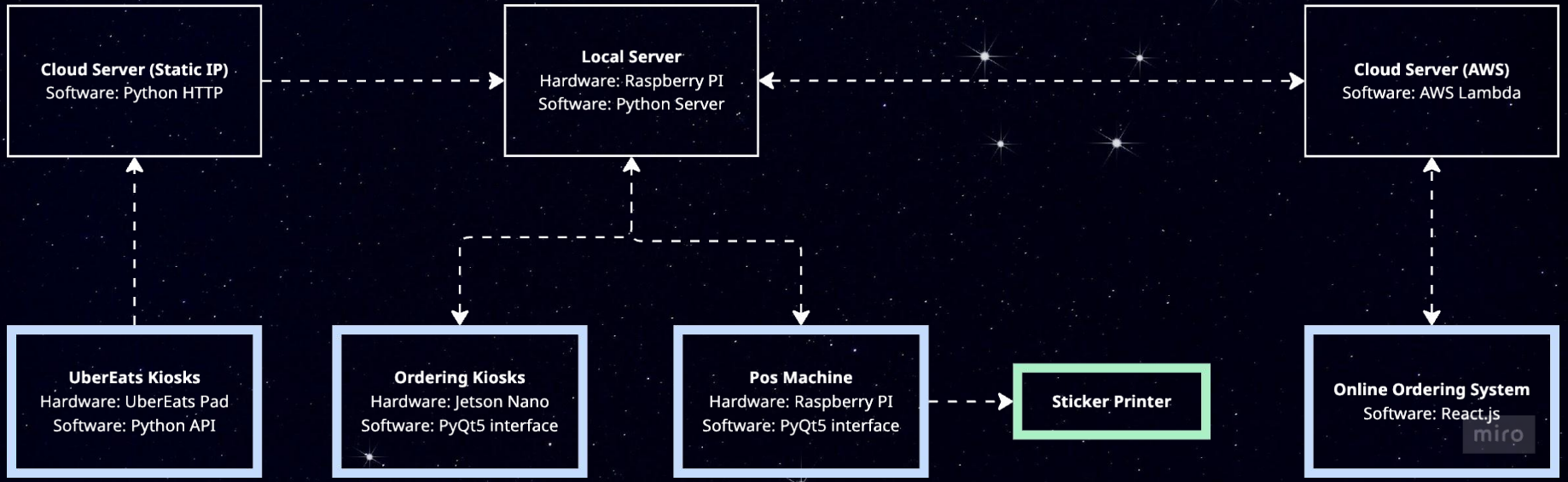
Goal: To build an automated system that reduces the need for staff and digitizes the ordering process.

Our Solution: 4 Order Pathways



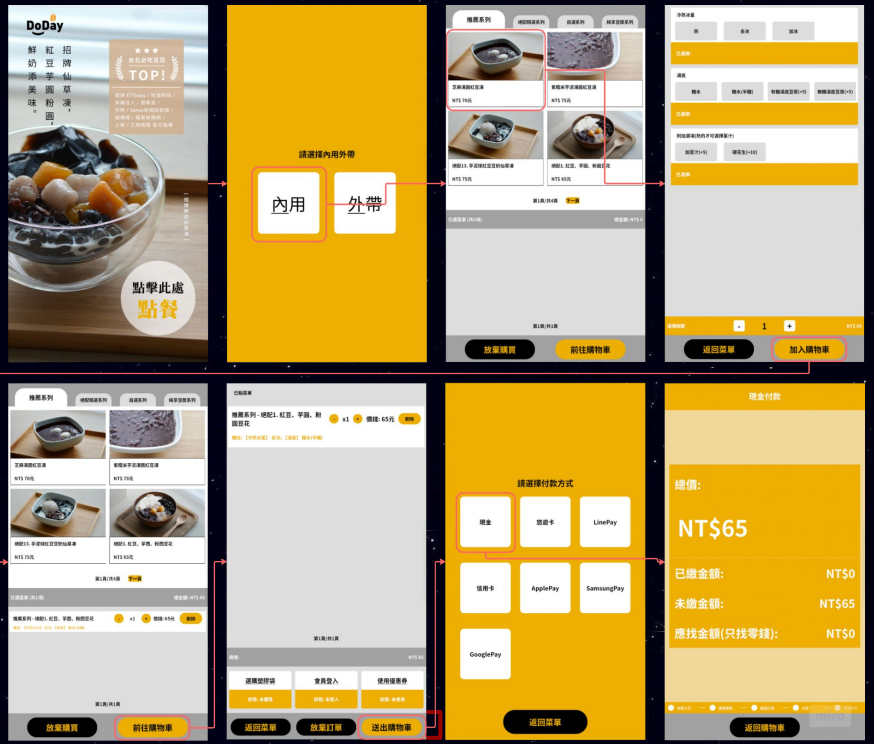
- **Kiosks** (Primary): Automates 90% of walk-in orders.
- **POS** (Fallback): Ensures system redundancy & edge-case handling.
- **UberEats** (Reach): Integrates 3rd-party delivery logistics.
- **Web App** (Margin): Bypasses 30% platform fees for bulk orders.
- **Mission: Route 4 distinct data streams into 1 physical sticker—zero manual entry.**

My Responsibilities



- Led a 5-person engineering team through a 3-year development cycle from concept to deployment.
- Directed the end-to-end system architecture, integrating hardware kiosks, edge servers, and AWS cloud infrastructure.
- Built the physical ordering machines, POS system, and online platform from scratch.
- Designed the hybrid API routing and successfully maintained the live production system for 6 continuous years.

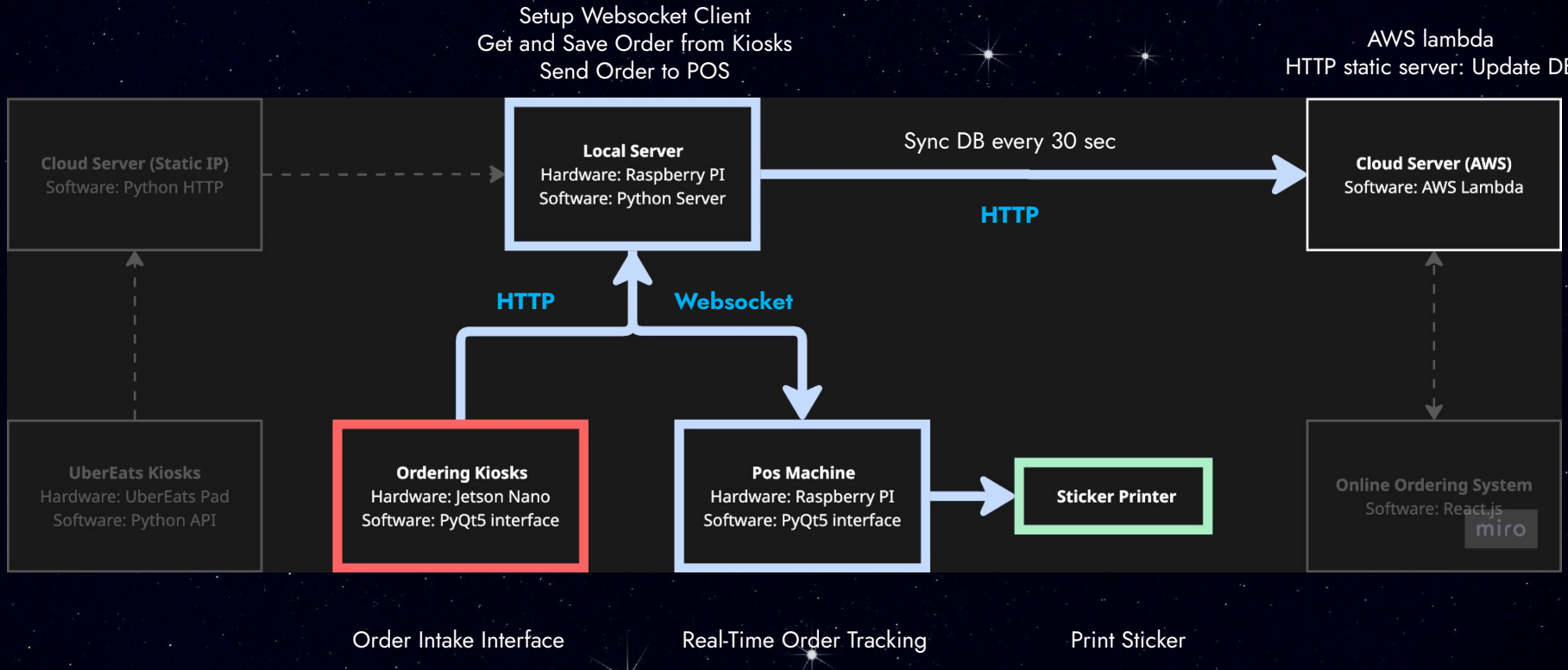
Path 1: Ordering Kiosks



submit_cart_function()

handle_special_day_and_add_to_order_api()

Path 1: Ordering Kiosks



Ordering Kiosks (Code)

proj-005-turkeypro_order_local_machine (doday-devel) - page_handler.py

```
def submit_cart_function(self):
        payment_method =
self.configs["payment_scheme"][self.page_payment_choice.sender().text(
)]
        # submit self.submit_data
        self.submit_data["payment_method"] = payment_method

        total price = int(self.submit data["total_price"])
        flag, message = self.post_verify_order()

        if total_price == 0:
            [# total_price == 0, don't need to jump into PaymentPage]
        else:
            if flag: [# if the order is valid]
            else: [# Error order, order again]


def handle_special_day_and_add_to_order_api(self):
        # split to single items for add to order
        self.submit_data.split_to_single_items()

        [# check if the customer orders any special day]

        if self.submit_data["spin_times"] <= 0:
            # if the customer don't order any special day
            self.post_add_to_order()
        else:
            # if the customer has ordered special day
            [# find special_day]

            # Call the next routine, start roulette
            self.roulette_alert_clock.start()
            self.special_day_display_wait_info(self.submit_data["spin_times"])
```

Post to Local Server to verify

Post the order to Local Server

Local Server (Code)

proj-005-cloud-server (doday-stable) - order_helper.py

```
def add_to_order_handler(input_request, **configs):
    # Import order data from json format
    new_order = OrderInfo.json_format_to_order_info(input_request)

    [# Handle order with promotion]
    [# Verify business hour and availability]

    # Handle order add for different order types
    order_type = new_order.get_order_type()
    new_order["machine_id"] = order_type

    with configs["db_lock"]:
        try:
            # Block the flow by at most 1 second. Avoid race condition
            with time limit(1):
                assign_new_id_and_add_order_to_db(new_order, **configs)

        except TimeoutException as e:
            logging.error("Timed out at assign_order_number and
add_to_order")

    [# Handle order with promotion]
    [# Update different new_order["payment_method"]]

    set_status_to_paid_and_send_to_websocket(new_order, **configs)

    return {"indicator": True, "message": "Order successfully added to
the system", "order_id": new_order["order_id"],
            "serial_number": new_order["serial_number"]}
```

Save Order to Local DB

Send Order to POS

POS Machine (Code)

proj-005-kitchen-queue-viewer (doday-stable) - QueuePage.py

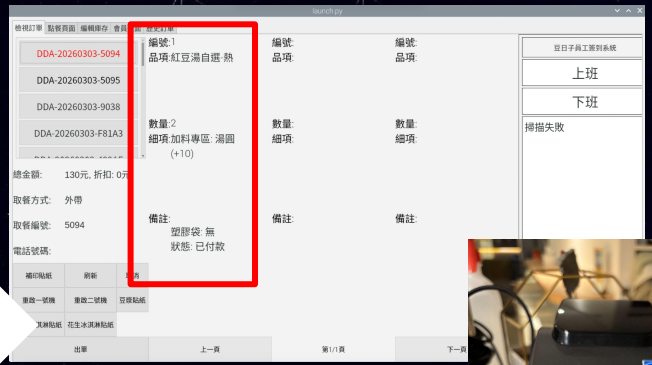
```
# Add qt signaling function to ws_client
self.wsclient.add_queue = dict_based_signal_connection()
self.wsclient.add_queue.signal.connect(self.add_queue)

...
def add_queue(self, order_info_dict):
    order_info = OrderInfo.json_format_to_order_info(order_info_dict)
    if order_info in self.queue_data and order_info["order_id"] in
self.queue_button_hashtable:
        return

    order_info_clone = copy.deepcopy(order_info)

    # Handler the case when we need sticker
    sticker = DodayOrderSticker(tmpfile_path=self.sticker_tmpfile_path,
**self.configs)
    deviceprint(self.sticker_printer_name,
self.sticker_tmpfile_path)(lambda order,**k:
sticker.order_info_to_order_sticker(order))(order_info_clone)

    order_info.merge_same_item()
    # Add the order to the queue
    self.queue_data.append(order_info)
    self.add_order_on_queue_layout(order_info)

    # Ring the order sound
    cli_pi_play_sound(self.ring_sound_path, 'hdmi', mode="normal")

    # Print Invoice
    # -----------------
    if self.printer_on:
        self.doday_api.order_print(order_info)
```

Print Sticker

Display the order on POS

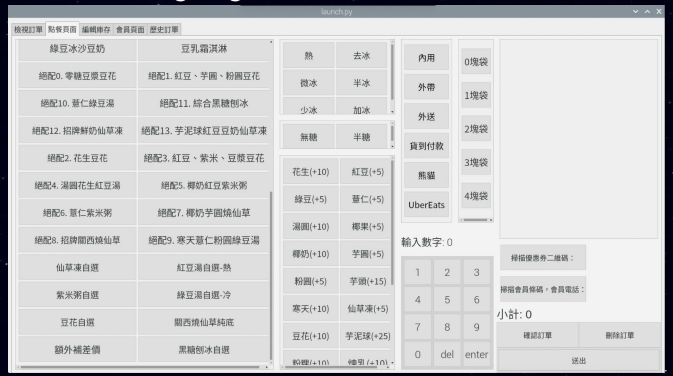
This is to print receipt

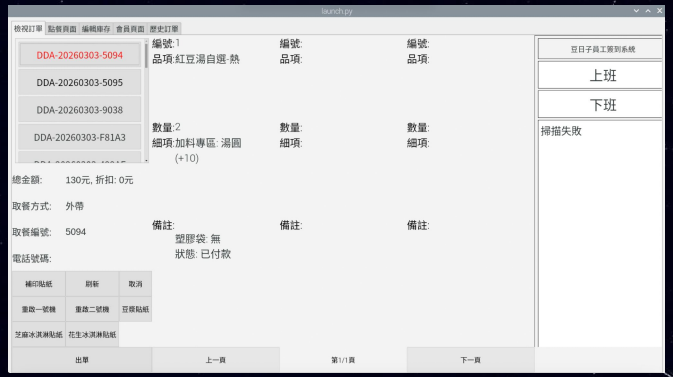
POS Machine



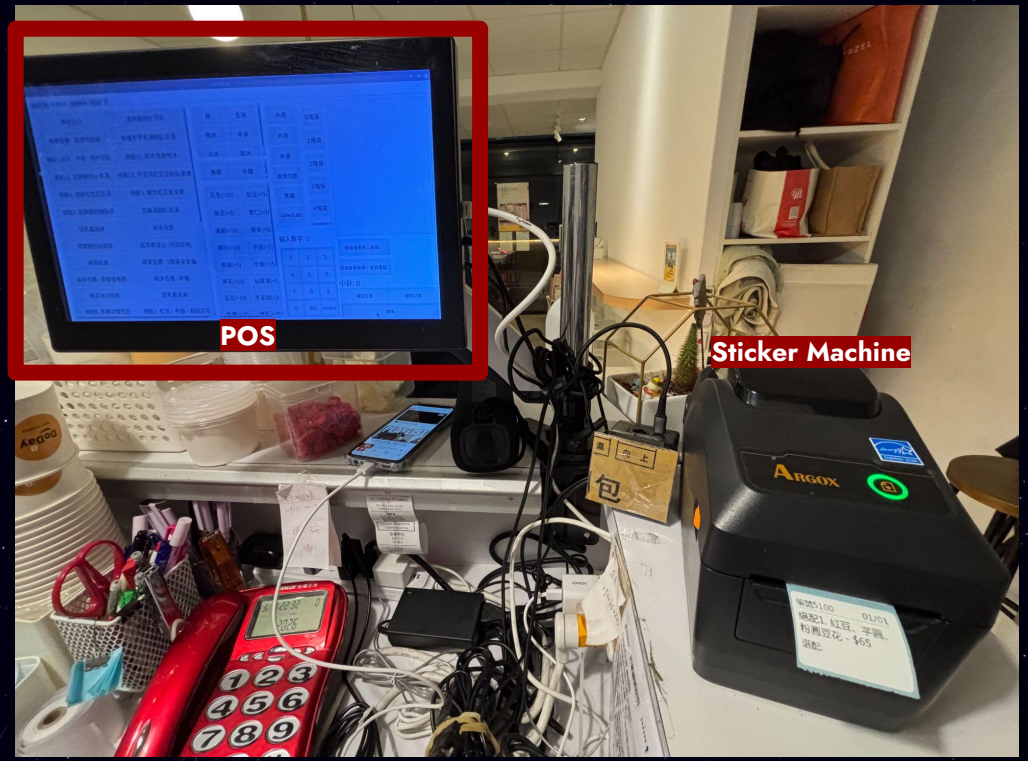
Path 2: POS Machine

POS Ordering Page

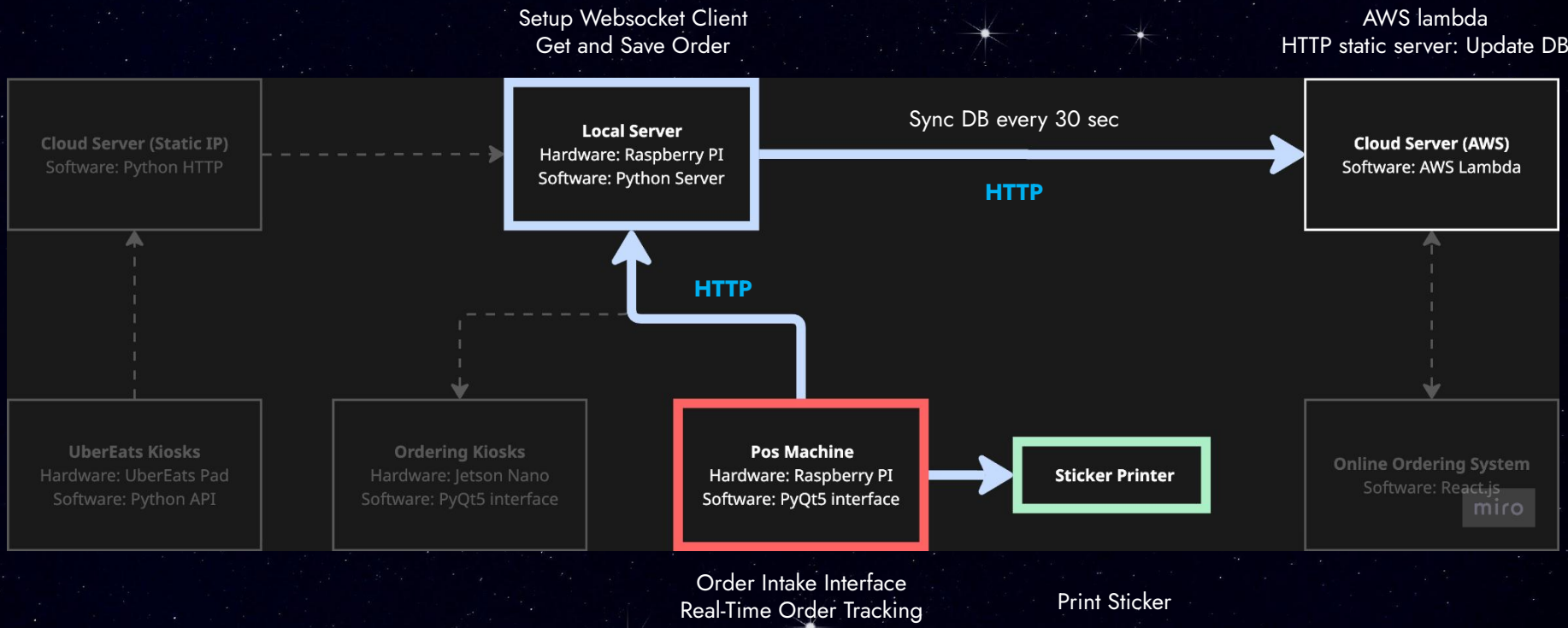




POS Order Display

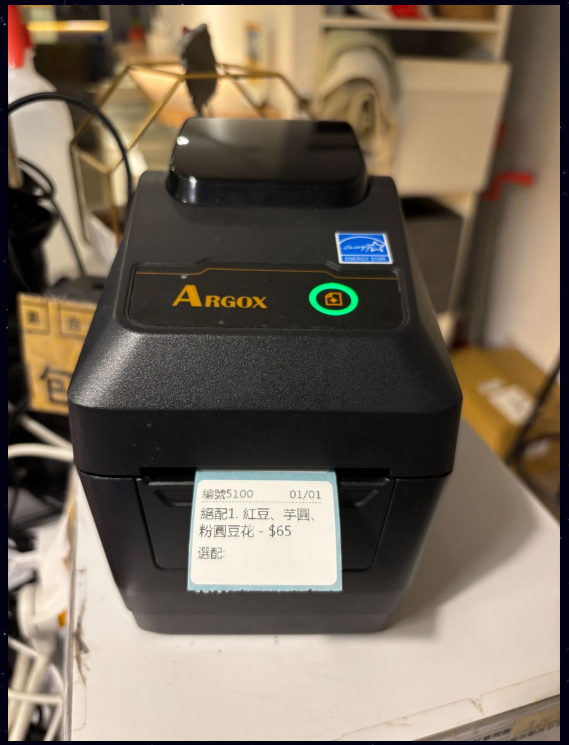


Path 2: POS Machine



POS Machine

Printer



POS Machine (Code)

proj-005-kitchen-queue-viewer (doday-stable) - KitchenOrderPage.py

```
def send_to_server(self, mod, **kwargs):
    if self.edit_order_status == 1:
        # Cannot send order to server if
        # it is in edit mode
        return False

    order_info = self.prepare_order_info()

    post_data = {
        "service": "order",
        "operation": "add_to_order",
    }
    post_data.update(order_info.tojson())

    ret = self.httpclient.client_post("main_server_http_url",
        post_data)
    res = json.loads(ret)

    if res["indicator"]:
        [# Reset Order Page UI]
    else:
        AlertToast(res["message"]).run()
```

Post the order to Local Server
(Same API with Ordering Kiosks)

Local Server (Database)

Latest Order

	_id [PK] integer	order_id text	order_no text	store_id text	serial_number text	username text	name1 text	name2 text	name3with4 text	stayortogo text	name5 text	amount integer	price text
1	382819	DDA-20260304-1001	2	DDA	1001		推薦系列	組配13. 芋泥球紅豆豆奶仙草凍	{"標準":["標準配料"]}	stay		1	75
2	382818	DDA-20260304-1001	1	DDA	1001		推薦系列	芝衡傳奇紅豆凍	{"標準":["標準配料"], "附加選...	stay		1	70
3	382817	DDA-20260304-5001	1	DDA	5001		組配精選系列	組配4. 滿滿花生紅豆湯	{"標準":["標準配料"], "附加選...	togo		1	70
4	382816	DDA-20260304-5000	1	DDA	5000		自選系列	紅豆湯自選-熱	{"加料專區":["芋圓(+5)","芋圓(...	togo		1	65
5	382815	DDA-20260304-7001	25	DDA	7001	None	組配精選系列	組配8. 招牌關西傳仙草	{"標準":["標準配料"]}	delivery		1	75
6	382814	DDA-20260304-7001	24	DDA	7001	None	組配精選系列	組配8. 招牌關西傳仙草	{"標準":["標準配料"]}	delivery		1	75
7	382813	DDA-20260304-7001	23	DDA	7001	None	組配精選系列	組配8. 招牌關西傳仙草	{"標準":["標準配料"]}	delivery		1	75
8	382812	DDA-20260304-7001	22	DDA	7001	None	組配精選系列	組配8. 招牌關西傳仙草	{"標準":["標準配料"]}	delivery		1	75
9	382811	DDA-20260304-7001	21	DDA	7001	None	組配精選系列	組配8. 招牌關西傳仙草	{"標準":["標準配料"]}	delivery		1	75
10	382810	DDA-20260304-7001	20	DDA	7001	None	組配精選系列	組配8. 招牌關西傳仙草	{"標準":["標準配料"]}	delivery		1	75

	price text	discount integer	final_price integer	total_price text	payment_method text	receipt_number text	status text	promotion text	promotion_key text	order_time integer	print_flag text	xml_flag text	comment1 text	comment2 text	comm text
	75	0	75	145	cash		preparing		TEST8	0		0	local-pay	0912	
	70	0	70	145	cash		preparing		TEST8	0		0	local-pay	0912	
		0	70	70	cash		preparing		TEST8	0		0	local-pay		
		0	65	65	cash		preparing		TEST8	0		0	local-pay		
	/5	0	75	1877	online-pay	None	preparing			0	None	None	2	online-pay	
	75	0	75	1877	online-pay	None	preparing			0	None	None	2	online-pay	
7	75	0	75	1877	online-pay	None	preparing			0	None	None	2	online-pay	
8	75	0	75	1877	online-pay	None	preparing			0	None	None	2	online-pay	
9	75	0	75	1877	online-pay	None	preparing			0	None	None	2	online-pay	
10	75	0	75	1877	online-pay	None	preparing			0	None	None	2	online-pay	

	comment3 text	comment4 text	comment5 text	comment6 text	comment7 text	comment8 text	time integer	sync_flag boolean
1	0912711214	{"pick_up_time": "", "phone_number": "", "delivery_location"...	{"50": 1, "10": 0, "5": 1, "1": 0, "100": 2, "200": 0, "500": 0}	not_updated	{"one_time_code": "", "deducted_point": 0}		1772599626	true
2	0912711214	{"pick_up_time": "", "phone_number": "", "delivery_location"...	{"50": 1, "10": 0, "5": 1, "1": 0, "100": 2, "200": 0, "500": 0}	not_updated	{"one_time_code": "", "deducted_point": 0}		1772599626	true
3		{"pick_up_time": "", "phone_number": "", "delivery_location"...	{"50": 0, "10": 3, "5": 0, "1": 0, "100": 1, "200": 0, "500": 0}	not_updated	{"one_time_code": "", "deducted_point": 0}		1772598634	true
4		{"pick_up_time": "", "phone_number": "", "delivery_location"...	{"50": -1, "10": -1, "5": -1, "1": 0, "100": 0, "200": 0, "500": ...	not_updated	{"one_time_code": "", "deducted_point": 0}		1772597316	true
5		{"pick_up_time": "03/04(三) 15:00", "phone_number": "096...	None	not_updated	{"one_time_code": "", "deducted_point": 0}	None	1772596978	true
6		{"pick_up_time": "03/04(三) 15:00", "phone_number": "096...	None	not_updated	{"one_time_code": "", "deducted_point": 0}	None	1772596978	true
7		{"pick_up_time": "03/04(三) 15:00", "phone_number": "096...	None	not_updated	{"one_time_code": "", "deducted_point": 0}	None	1772596978	true
8		{"pick_up_time": "03/04(三) 15:00", "phone_number": "096...	None	not_updated	{"one_time_code": "", "deducted_point": 0}	None	1772596978	true
9		{"pick_up_time": "03/04(三) 15:00", "phone_number": "096...	None	not_updated	{"one_time_code": "", "deducted_point": 0}	None	1772596978	true
10		{"pick_up_time": "03/04(三) 15:00", "phone_number": "096...	None	not_updated	{"one_time_code": "", "deducted_point": 0}	None	1772596978	true

Sync flag

Local Server (Code)

proj-005-cloud-server (doday-stable) - gados_order_sync.py

```
def gados_order_sync(**configs):
    with configs["sync_lock"]:
        # lock the entire sync and update process
        unsynced_dict = get_not_synced_dict()

        ret = json.loads(post_order_sync_to_cloud({"store_id":
configs["store_id"], "brand_id": configs["brand_id"],
                "unsynced_dict": unsynced_dict}, **configs))

        if ret.get("indicator", False):
            update_local_sync_status(unsynced_dict)

        if isinstance(ret.get("message", False), dict) and
ret.get("message", {}).get("new_orders_from_online", False):

add_order_to_local_server(ret["message"]["new_orders_from_online"],
**configs)

        if isinstance(ret.get("message", False), dict) and
ret.get("message", {}).get("store_info_from_online", False):

update_store_information_to_local_server(ret["message"]["store_info_fr
om_online"], **configs)

        if isinstance(ret.get("message", False), dict) and
ret.get("message", {}).get("promotion_info_from_online", False):

update_promotion_to_local_server(ret["message"]["promotion_info_from_o
nline"], **configs)
```

Sync Local to Cloud

AWS Server

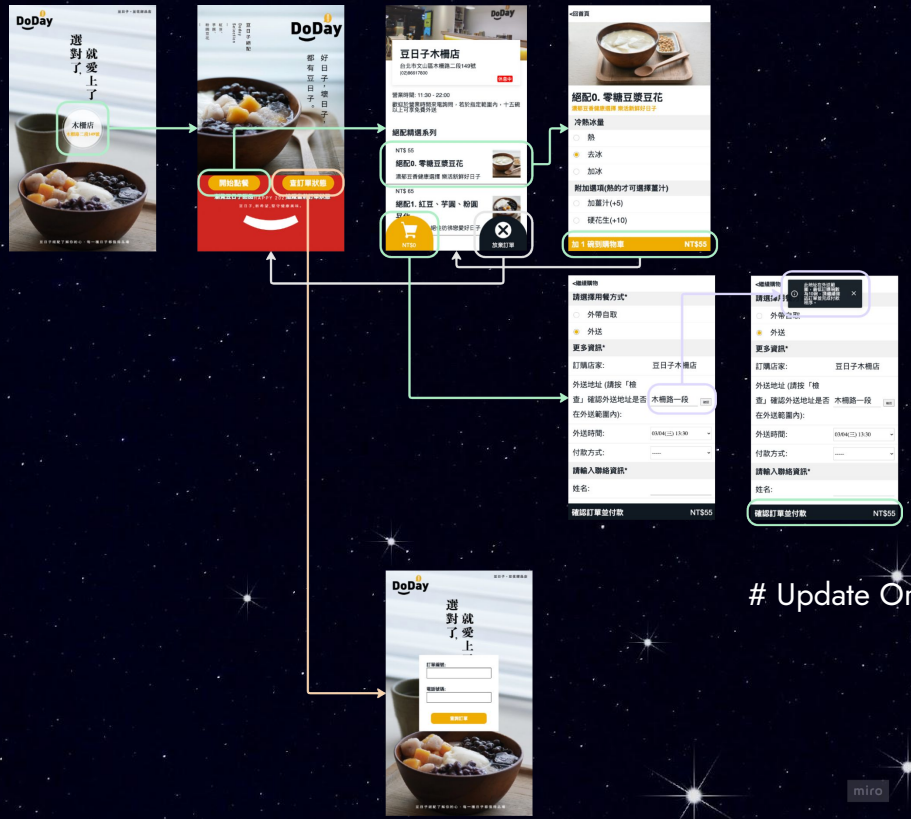
PCubeOnlineOrderServer (doday-stable) - order_helper.py

```
def post_order_sync_to_cloud_handler(input_request, **configs):
    store_id, unsynced_dict, brand_id = input_request['store_id'], \
        input_request['unsynced_dict'], input_request["brand_id"]

    [# First check whether the order id is in the database or not]
    data = get_data('order',['order_id'],where_value_list, multiple=True)
    [# Handle the existing order list and only update the flag]

    existing_order_list_clone = copy.deepcopy(existing_order_list)
    for order_clone in existing_order_list_clone:
        order_clone["sync_flag"] = True

    _ = edit_on_table("order",existing_order_list_clone,["order_id","order_no"])

    [# Handle the none existing order and add those to the table]

    _ = add_to_table("order", add_list)

    [# Update promotion key usage and member related info]

    # Future Modification
    message_dict = {}
    # After syncing the data from local, try to get those from the
    # cloud db that haven't been synced.
    not_synced_cloud_order = get_not_synced_dict(store_id)
    if not_synced_cloud_order == {}:
        # Empty unsynced order
        message_dict.update({"new_orders_from_online": False})
    else:
        message_dict.update({"new_orders_from_online": not_synced_cloud_order})

    return {"indicator":True,"message":message_dict}
```

Save Order to DB

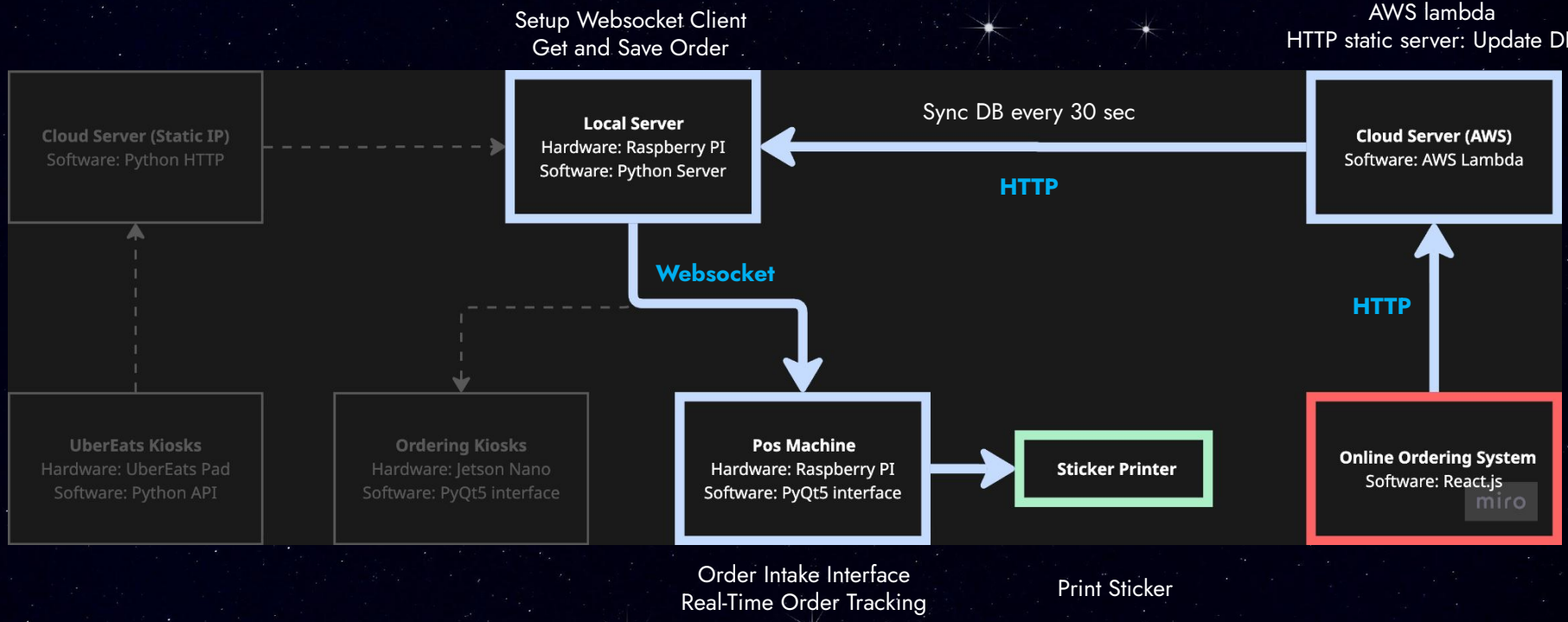
Send not Synced Order to local

Path 3: Online Ordering System



Update Order to AWS Server

Path 3: Online Ordering System



Online Ordering (Code)

AWS Server (Database)

proj-005-react_online_order (doday-devel) - cart_page.js

```
async handleSubmit(e) {
 if {[// check order]}
 else{
   e.target.disabled = true
   [// check member]
   var order_list_data = getCookie("shoppingCartItems");
   var decode_list = await get_cart_item(decode_order_list)
   [// revise format]
   [// Repeatedly add the same data for the same bowl, for number of bowls]

   var request_data = {
     "service": "order",
     "operation": "add_to_order",
     "name1": category_list,
     "name2": item_list,
     "name3with4": addon_list,
     "name5": comment_list,
     "number_of_data": number_of_bowl.toString(),
     "price": price_list,
     "total_price": this.state.totalPrice.toString(),
     "promotion_key": coupon_fee,
     "final_price": price_list,
     "payment_method": online_or_local(this.state.stayOrTogo,
this.state.extraInfo["payment_method"]),
     "store_id": getCookie("store_id"),
   }

   var request_str = JSON.stringify(request_data)
   postDataFromServer(process.env.REACT_APP_SERVER_URL + '?', request_str)
     .then(result => {[// update web page]})
     .catch(error => {[// switch to error page]})
 }
}
```

Send Order to AWS

Latest Order

	_id [PK] integer	order_id text	order_no text	store_id text	serial_number text	username text	name1 text	name2 text	name3with4 text	stayortogo text	name5 text	amount integer	price text
1	382819	DDA-20260304-1001	2	DDA	1001		推薦系列	綜配13. 芋泥球紅豆豆奶仙草凍	{"標準":["標準配料"]}	stay		1	75
2	382818	DDA-20260304-1001	1	DDA	7001		經典系列	芝衡傳奇紅豆凍	{"標準":["標準配料"], "附加選...	stay		1	70
3	382817	DDA-20260304-5001	1	DDA	5001		綜配精選系列	綜配4. 漫園花生紅豆湯	{"標準":["標準配料"], "附加選...	togo		1	70
4	382816	DDA-20260304-5000	1	DDA	5000		自選系列	紅豆湯自選-熱	{"加料專區":{"芋圓(+5)","芋圓(...	togo		1	65
5	382815	DDA-20260304-7001	25	DDA	7001	None	綜配精選系列	綜配8. 招牌關西傳仙草	{"標準":["標準配料"]}	delivery		1	75
6	382814	DDA-20260304-7001	24	DDA	7001	None	綜配精選系列	綜配8. 招牌關西傳仙草	{"標準":["標準配料"]}	delivery		1	75
7	382813	DDA-20260304-7001	23	DDA	7001	None	綜配精選系列	綜配8. 招牌關西傳仙草	{"標準":["標準配料"]}	delivery		1	75
8	382812	DDA-20260304-7001	22	DDA	7001	None	綜配精選系列	綜配8. 招牌關西傳仙草	{"標準":["標準配料"]}	delivery		1	75
9	382811	DDA-20260304-7001	21	DDA	7001	None	綜配精選系列	綜配8. 招牌關西傳仙草	{"標準":["標準配料"]}	delivery		1	75
10	382810	DDA-20260304-7001	20	DDA	7001	None	綜配精選系列	綜配8. 招牌關西傳仙草	{"標準":["標準配料"]}	delivery		1	75

	price text	discount integer	final_price integer	total_price integer	payment_method text	receipt_number text	status text	promotion text	promotion_key text	order_time integer	print_flag text	xml_flag text	comment1 text	comment2 text	comm text
	75	0	75	145	cash		preparing		TEST8	0		0	local-pay	0912	
	70	0	70	145	cash		preparing		TEST8	0		0	local-pay	0912	
		0	70	70	cash		preparing		TEST8	0		0	local-pay		
		0	65	65	cash		preparing		TEST8	0		0	local-pay		
	75	0	75	1877	online-pay	None	preparing			0	None	None	2	online-pay	
	75	0	75	1877	online-pay	None	preparing			0	None	None	2	online-pay	
7	75	0	75	1877	online-pay	None	preparing			0	None	None	2	online-pay	
8	75	0	75	1877	online-pay	None	preparing			0	None	None	2	online-pay	
9	75	0	75	1877	online-pay	None	preparing			0	None	None	2	online-pay	
10	75	0	75	1877	online-pay	None	preparing			0	None	None	2	online-pay	

	comment3 text	comment4 text	comment5 text	comment6 text	comment7 text	comment8 text	time integer	sync_flag boolean
1	0912711214	{"pick_up_time": "", "phone_number": "", "delivery_location":...	{"50": 1, "10": 0, "5": 1, "1": 0, "100": 2, "200": 0, "500": 0}	not_updated	{"one_time_code": "", "deducted_point": 0}		1772599626	true
2	0912711214	{"pick_up_time": "", "phone_number": "", "delivery_location":...	{"50": 1, "10": 0, "5": 1, "1": 0, "100": 2, "200": 0, "500": 0}	not_updated	{"one_time_code": "", "deducted_point": 0}		1772599626	true
3		{"pick_up_time": "", "phone_number": "", "delivery_location":...	{"50": 0, "10": 3, "5": 0, "1": 0, "100": 1, "200": 0, "500": 0}	not_updated	{"one_time_code": "", "deducted_point": 0}		1772598634	true
4		{"pick_up_time": "", "phone_number": "", "delivery_location":...	{"50": -1, "10": -1, "5": -1, "1": 0, "100": 0, "200": 0, "500": ...	not_updated	{"one_time_code": "", "deducted_point": 0}		1772597316	true
5		{"pick_up_time": "03/04(三) 15:00", "phone_number": "096...	None	not_updated	{"one_time_code": "", "deducted_point": 0}	None	1772596978	true
6		{"pick_up_time": "03/04(三) 15:00", "phone_number": "096...	None	not_updated	{"one_time_code": "", "deducted_point": 0}	None	1772596978	true
7		{"pick_up_time": "03/04(三) 15:00", "phone_number": "096...	None	not_updated	{"one_time_code": "", "deducted_point": 0}	None	1772596978	true
8		{"pick_up_time": "03/04(三) 15:00", "phone_number": "096...	None	not_updated	{"one_time_code": "", "deducted_point": 0}	None	1772596978	true
9		{"pick_up_time": "03/04(三) 15:00", "phone_number": "096...	None	not_updated	{"one_time_code": "", "deducted_point": 0}	None	1772596978	true
10		{"pick_up_time": "03/04(三) 15:00", "phone_number": "096...	None	not_updated	{"one_time_code": "", "deducted_point": 0}	None	1772596978	true

Sync flag

AWS Server (Code)

PCubeOnlineOrderServer (doday-stable) - order_helper.py

```
def post_order_sync_to_cloud_handler(input_request, **configs):
    store_id, unsynced_dict, brand_id = input_request['store_id'], \
        input_request['unsynced_dict'], input_request["brand_id"]

    [# First check whether the order id is in the database or not]
    data = get_data('order',['order_id'],where_value_list, multiple=True)
    [# Handle the existing order list and only update the flag]

    existing_order_list_clone = copy.deepcopy(existing_order_list)
    for order_clone in existing_order_list_clone:
        order_clone["sync_flag"] = True

    _ = edit_on_table("order",existing_order_list_clone,["order_id","order

    [# Handle the none existing order and add those to the table]

    _ = add_to_table("order", add_list)

    [# Update promotion key usage and member related info]

    # Future Modification
    message_dict = {}
    # After syncing the data from local, try to get those from the
    # cloud db that haven't been synced.
    not_synced_cloud_order = get_not_synced_dict(store_id)
    if not_synced_cloud_order == {}:
        # Empty unsynced order
        message_dict.update({"new_orders_from_online": False})
    else:
        message_dict.update({"new_orders_from_online": not_synced_cloud_order})

    return {"indicator":True,"message":message_dict}
```

Save Order to DB

Send not Synced Order to local

Local Server (Code)

proj-005-cloud-server (doday-stable) - gados_order_sync.py

```
def gados_order_sync(**configs):
    with configs["sync_lock"]:
        # lock the entire sync and update process
        unsynced_dict = get_not_synced_dict()
        logging.info("[Gados Order Sync] Syncing:
"+str(list(unsynced_dict.keys())))
        ret = json.loads(post_order_sync_to_cloud({"store_id":
configs["store_id"], "brand_id": configs["brand_id"],
                "unsynced_dict": unsynced_dict}, **configs))

        if ret.get("indicator", False):
            update_local_sync_status(unsynced_dict)

        if isinstance(ret.get("message", False), dict) and
ret.get("message", {}).get("new_orders_from_online", False):

add_order_to_local_server(ret["message"]["new_orders_from_online"],
**configs)

        if isinstance(ret.get("message", False), dict) and
ret.get("message", {}).get("store_info_from_online", False):

update_store_information_to_local_server(ret["message"]["store_info_fr
om_online"], **configs)

        if isinstance(ret.get("message", False), dict) and
ret.get("message", {}).get("promotion_info_from_online", False):

update_promotion_to_local_server(ret["message"]["promotion_info_from_o
nline"], **configs)
```

Sync Cloud to Local

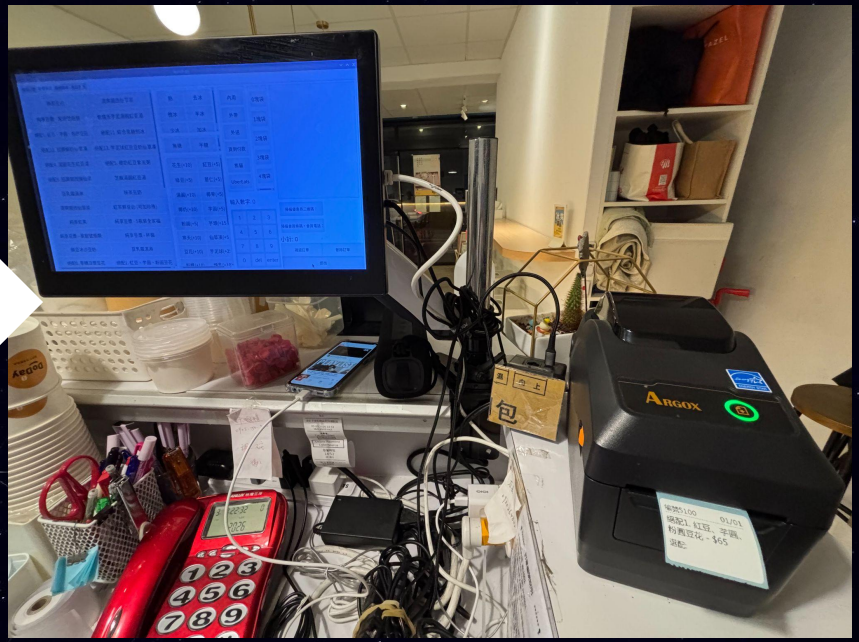
Local Server (Code)

proj-005-cloud-server (doday-stable) - gados_order_sync.py

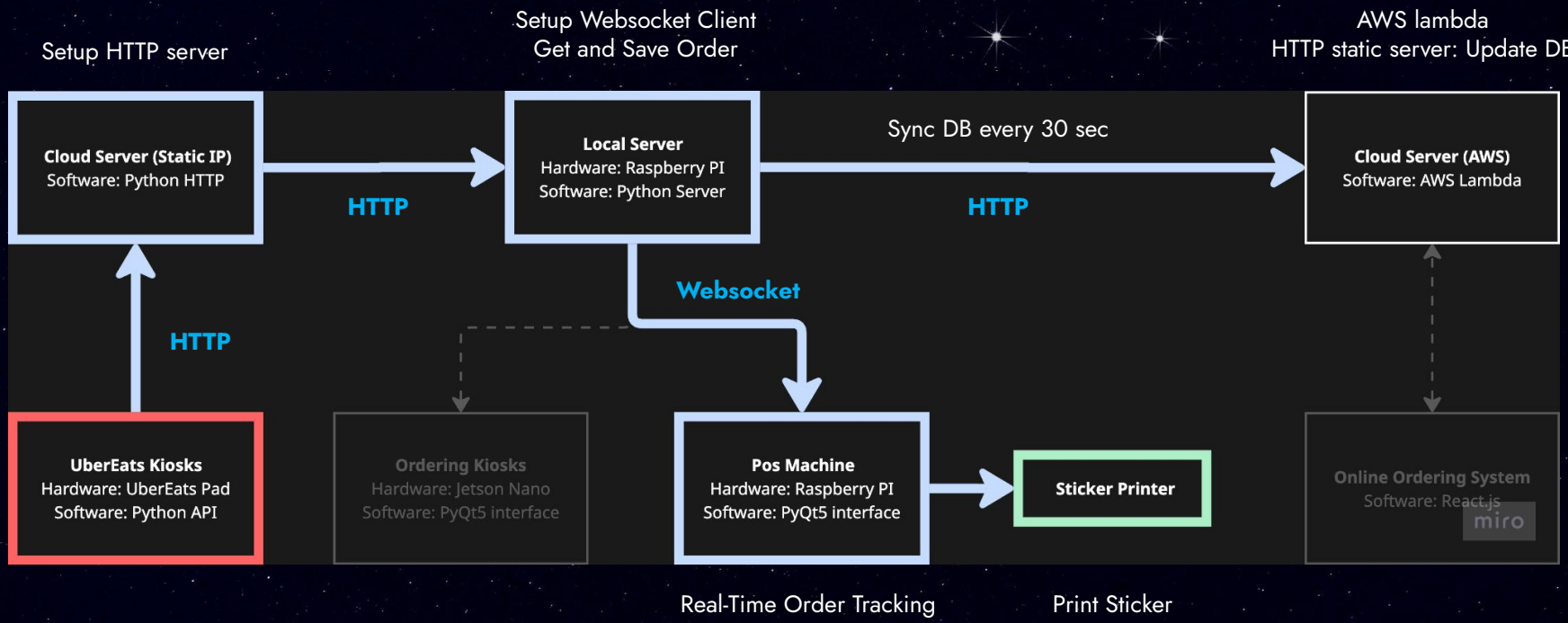
```
def add_order_to_local_server(payload, **configs):
    order_id_list = list(payload.keys())
    where_value_list = []
    for oid in order_id_list:
        where_value_list.append((oid,))

    matched_data = get_data("order", ["order_id"], where_value_list,
multiple=True)
    exist_order_id_set = set()
    for data in matched_data:
        exist_order_id_set.add(data["order_id"])

    for order_id in payload:
        if order_id in exist_order_id_set:
            continue
        order_info = OrderInfo.db_format_to_order_info(payload[order_id])
        order_info.sync_flag = True
        dict_data = order_info.tojson()
        dict_data.update({"service": "order", "operation": "add_to_order"})
        client = GadgetHiClient(**configs)
        _ = client.client_post("local_server_http_url", dict_data)
```

Post the order to Local Server
(Same API with Ordering Kiosks)

POS Machine



Path 4: UberEats



Cloud Server (Code)

proj-002-uber_eats_integration (doday-stable) - server.py

```
def main_handler(cls, event, **configs):
    uberEatsManager = UberEats(configs["uber_client_id"],
configs["uber_client_secret"], store_id,
configs["uber_eats_secret_filepath"])

    [# Verify whether the information comes from Uber]

    [# change the details to orderinfo]
    order_details = uberEatsManager.get_order_details(order_id)
    uber_order_info = uber_eats_order_to_order_info(order_details,
uber_mapping)

    post_data = {
        "service": "order",
        "operation": "uber_add_to_order",
    }
    post_data.update(uber_order_info.tojson())

    if store_id in uber_store_id_to_doday_store_id_map.keys():
        client = DodayHttpClient(**configs)
        ret = client.client_post("uber_http_url", post_data, gauth=True,
gadgethi_key=configs["gadgethi_key"],gadgethi_secret=configs["gadgethi_se
cret"])
        res = json.loads(ret)

    return {'statusCode': 200, 'body': 'Successful from Uber Server'}
```

Local Server (Database)

Latest Order

	_id [PK] integer	order_id text	order_no text	store_id text	serial_number text	username text	name1 text	name2 text	name3with4 text	stayortogo text	name5 text	amount integer	price text
1	382819	DDA-20260304-1001	2	DDA	1001		推薦系列	組配13. 芋泥球紅豆豆奶仙草凍	{"標準":["標準配料"]}	stay		1	75
2	382818	DDA-20260304-1001	1	DDA	1001		推薦系列	芝麻滷蛋紅豆湯	{"標準":["標準配料"],"附加選...	stay		1	75
3	382817	DDA-20260304-5001	1	DDA	5001		組配精選系列	組配4. 滷蛋花生紅豆湯	{"標準":["標準配料"],"附加選...	togo		1	70
4	382816	DDA-20260304-5000	1	DDA	5000		自選系列	紅豆湯自選-熱	{"加料專區":["芋圓(+5)","芋圓(...	togo		1	65
5	382815	DDA-20260304-7001	25	DDA	7001	None	組配精選系列	組配8. 招牌關西傳仙草	{"標準":["標準配料"]}	delivery		1	75
6	382814	DDA-20260304-7001	24	DDA	7001	None	組配精選系列	組配8. 招牌關西傳仙草	{"標準":["標準配料"]}	delivery		1	75
7	382813	DDA-20260304-7001	23	DDA	7001	None	組配精選系列	組配8. 招牌關西傳仙草	{"標準":["標準配料"]}	delivery		1	75
8	382812	DDA-20260304-7001	22	DDA	7001	None	組配精選系列	組配8. 招牌關西傳仙草	{"標準":["標準配料"]}	delivery		1	75
9	382811	DDA-20260304-7001	21	DDA	7001	None	組配精選系列	組配8. 招牌關西傳仙草	{"標準":["標準配料"]}	delivery		1	75
10	382810	DDA-20260304-7001	20	DDA	7001	None	組配精選系列	組配8. 招牌關西傳仙草	{"標準":["標準配料"]}	delivery		1	75

	price text	discount integer	final_price integer	total_price integer	payment_method text	receipt_number text	status text	promotion text	promotion_key text	order_time integer	print_flag text	xml_flag text	comment1 text	comment2 text	comm text
	75	0	75	145	cash		preparing		TEST8	0		0	local-pay	0912	
	70	0	70	145	cash		preparing		TEST8	0		0	local-pay	0912	
	70	0	70	70	cash		preparing		TEST8	0		0	local-pay		
	65	0	65	65	cash		preparing		TEST8	0		0	local-pay		
	75	0	75	1877	online-pay	None	preparing			0	None	None	2	online-pay	
	75	0	75	1877	online-pay	None	preparing			0	None	None	2	online-pay	
7	75	0	75	1877	online-pay	None	preparing			0	None	None	2	online-pay	
8	75	0	75	1877	online-pay	None	preparing			0	None	None	2	online-pay	
9	75	0	75	1877	online-pay	None	preparing			0	None	None	2	online-pay	
10	75	0	75	1877	online-pay	None	preparing			0	None	None	2	online-pay	

	comment3 text	comment4 text	comment5 text	comment6 text	comment7 text	comment8 text	time integer	sync_flag boolean
1	0912711214	{"pick_up_time": "", "phone_number": "", "delivery_location":...	{"50": 1, "10": 0, "5": 1, "1": 0, "100": 2, "200": 0, "500": 0}	not_updated	{"one_time_code": "", "deducted_point": 0}		1772599626	true
2	0912711214	{"pick_up_time": "", "phone_number": "", "delivery_location":...	{"50": 1, "10": 0, "5": 1, "1": 0, "100": 2, "200": 0, "500": 0}	not_updated	{"one_time_code": "", "deducted_point": 0}		1772599626	true
3		{"pick_up_time": "", "phone_number": "", "delivery_location":...	{"50": 0, "10": 3, "5": 0, "1": 0, "100": 1, "200": 0, "500": 0}	not_updated	{"one_time_code": "", "deducted_point": 0}		1772598634	true
4		{"pick_up_time": "", "phone_number": "", "delivery_location":...	{"50": -1, "10": -1, "5": -1, "1": 0, "100": 0, "200": 0, "500": ...	not_updated	{"one_time_code": "", "deducted_point": 0}		1772597316	true
5		{"pick_up_time": "03/04(三) 15:00", "phone_number": "096...	None	not_updated	{"one_time_code": "", "deducted_point": 0}	None	1772596978	true
6		{"pick_up_time": "03/04(三) 15:00", "phone_number": "096...	None	not_updated	{"one_time_code": "", "deducted_point": 0}	None	1772596978	true
7		{"pick_up_time": "03/04(三) 15:00", "phone_number": "096...	None	not_updated	{"one_time_code": "", "deducted_point": 0}	None	1772596978	true
8		{"pick_up_time": "03/04(三) 15:00", "phone_number": "096...	None	not_updated	{"one_time_code": "", "deducted_point": 0}	None	1772596978	true
9		{"pick_up_time": "03/04(三) 15:00", "phone_number": "096...	None	not_updated	{"one_time_code": "", "deducted_point": 0}	None	1772596978	true
10		{"pick_up_time": "03/04(三) 15:00", "phone_number": "096...	None	not_updated	{"one_time_code": "", "deducted_point": 0}	None	1772596978	true

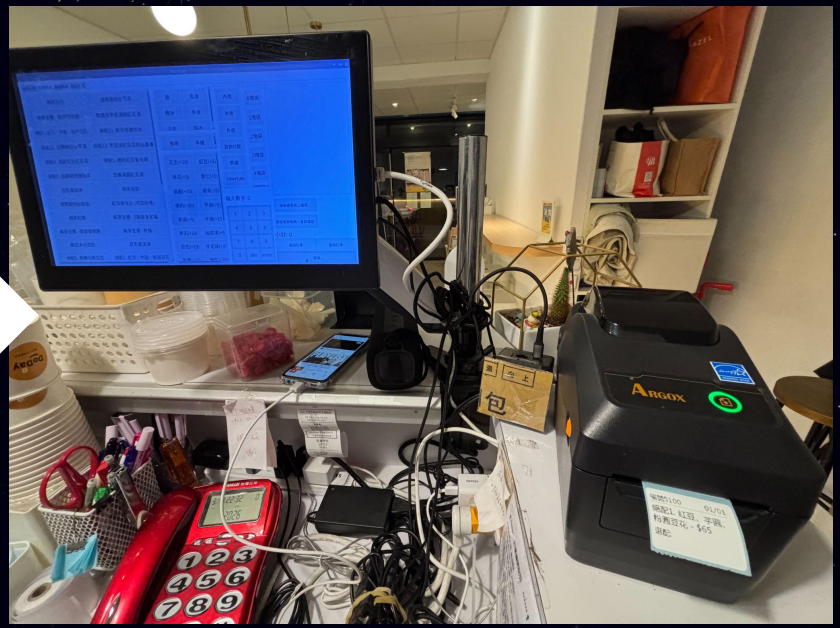
Sync flag

Local Server (Code)

proj-005-cloud-server (doday-stable) - server.py

```
def uber_add_to_order_handler(input_request, **configs):
    new_order = OrderInfo.json_format_to_order_info(input_request)

    [# Verify availability]

    # Check if it satisfies uber order convention
    assert "UBER-" in new_order["order_id"]
    uber_display_id = new_order["order_id"].replace("UBER-", "")

    today_date = datetime.date.today().strftime("%Y%m%d")

    assigned_order_id =
configs["store_id"]+'-'+today_date+'-'+uber_display_id

    # assign id and no to order info
    new_order.assign_order_no()
    new_order["store_id"] = configs["store_id"]
    new_order["order_id"] = assigned_order_id
    new_order["serial_number"] = "u"+uber_display_id

    # Add order to database (order table)
    _ = add_to_table('order',new_order.tosplitted())

    # Turn this on if we want to notify local pos machine and TV.
    set_status_to_paid_and_send_to_websocket(new_order, **configs)
    indicator, message = True, "Order successfully added to the system"

    return {"indicator": indicator, "message": message, "orderid":
new_order["order_id"]}
```

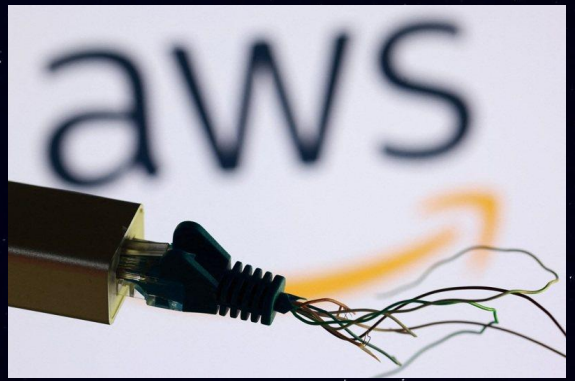
POS Machine



Challenges

1. Unpredictable consumer behaviors and edge cases across 4 distinct ordering methods.
2. Over the past 6 years, we have experienced AWS network outages, hardware jams, power loss, and physical cable damage (e.g., damage by mice).

Software must be fault-tolerant because the physical world is unpredictable.



Challenges

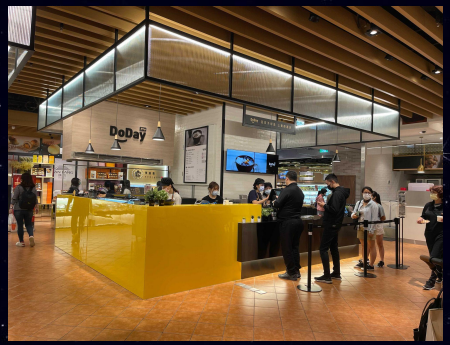
1. **Unpredictable consumer behaviors and edge cases across 4 distinct ordering methods.**

- Leveraging comprehensive logging and behavioral data collection to design a more inclusive and resilient system.

2. **Over the past 6 years, we have experienced AWS network outages, hardware jams, power loss, and physical cable damage (e.g., damage by mice).**

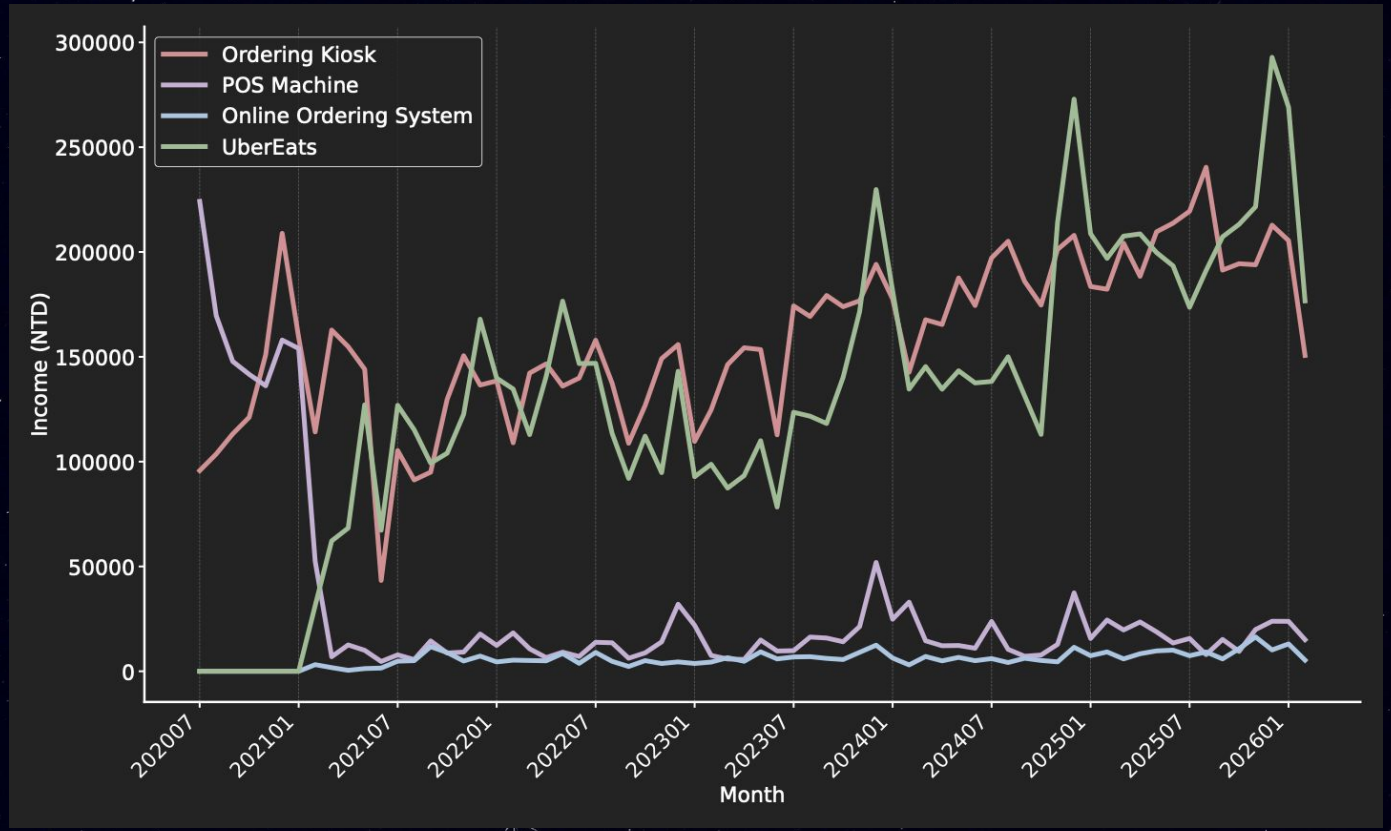
- **AWS network outages:** local/cloud design; if the cloud server fails, the local system can still function.
- **Hardware Jams:** modularize hardware parts; make it easy to replace when a part is broken.
- **Power Loss:** local/cloud server design; ensure data won't be lost when power is suddenly lost.
- **Physical Cable Damage:** modularize hardware replacement; system logs display the status of each hardware module.

Results



Successfully supported the expansion of 3 retail stores over 6 years with highly stable system performance.

Results


Ordering Kiosk
POS Machine
Online Ordering System
UberEats
Income (NTD)
300000
250000
200000
150000
100000
50000
0
202007
202101
202107
202201
202207
202301
202307
202401
202407
202501
202507
202601
Month

Lessons Learned

- The real world is complicated; no matter how hard we try to include all possibilities in our system, there will always be some unexpected scenarios. How to make sure our system does not fail catastrophically is important.
- Automation is hard because humans are using it. If the world were just robots, it would be easy, but humans are unpredictable; we need to use these things to help humans without generating more problems.
- How to make a product rather than an academic project.